TENANTPRO — SMART TENANT AND RENT MANAGEMENT SYSTEM



A project report submitted in partial fulfilment of the requirements of
Master of Computer Applications Post Graduate Course of
JAIN (Deemed-to-be University)

Submitted by

Anush Kulal M **24MCAR0133**
Akshita Bansal **24MCAR0131**
Chandan Mathur **24MCAR0230**

Under the guidance of

Dr. Kamalraj R

Department of Computer Science and IT
JAIN (Deemed-to-be University)
Bengaluru

Date -

CERTIFICATE BY THE UNIVERSITY

Certified that the project report titled “TenantPro — Smart Tenant and Rent Management System” is based on an original project study conducted by Anush Kulal M (Register No. 24MCAR0133), Akshita Bansal (Register No. 24MCAR0131), Chandan Mathur (Register No. 24MCAR0230) of the Master of Computer Applications programme, under the guidance of Dr. Kamalraj R. They have attended the required guidance sessions held. The project report has not formed a basis for the award of any Degree or Diploma of any university or any other institution.

PLACE: Bengaluru

DATE:

Dr. SAGAR GULATI
DIRECTOR
SCHOOL OF COMPUTER SCIENCE AND IT
JAIN (Deemed-to-be University)

CERTIFICATE BY THE GUIDE

Certified that the project report titled “TenantPro — Smart Tenant and Rent Management System” is based on an original project study conducted by Anush Kulal M (Register No. 24MCAR0133), Akshita Bansal (Register No. 24MCAR0131), Chandan Mathur (Register No. 24MCAR0230) under my guidance. This project has not formed a basis for the award of any Degree or Diploma of any University or any other Institution.

PLACE: Bengaluru

DATE:

Signature of the Guide

Dr. Kamalraj R

Department of Computer Science and IT

DECLARATION BY THE STUDENTS

This is to declare that the project titled “TenantPro — Smart Tenant and Rent Management System” is carried out independently by us under the guidance of Dr. Kamalraj R, Department of Computer Science and IT, JAIN (Deemed-to-be University), Bengaluru. This work is an original one and has not been submitted earlier to any University or any other Institution for the fulfilment of the requirement of a course of study or any other credential.

PLACE: Bengaluru

DATE:

NAME: Anush Kulal M
USN: 24MCAR0133

NAME: Akshita Bansal
USN: 24MCAR0131

NAME: Chandan Mathur
USN: 24MCAR0230

ACKNOWLEDGEMENT

We are deeply indebted to Dr. CHENRAJ ROYCHAND, President, JAIN (Deemed-to-be University) Trust, Bengaluru, for having admitted us to undergo the Master of Computer Applications Post Graduate course during the academic term 2025 - 2026.

We are pleased to thank Dr. Sagar Gulati, Director, School of Computer Science and IT, JAIN (Deemed-to-be University), Bengaluru, for the facilities and the academic environment extended to us during the course of this work.

We take this opportunity to express our gratitude to Dr. Ananta Charan Ojha, Deputy Director, Dr. K Suneetha, Head of the Department of Computer Science and IT, and Dr. Murugan R, Programme Head of the MCA Programme, for their valuable guidance and support for the successful completion of the project work.

We express our profound thanks to our project coordinator Dr. I Brem Navas, Professor, Department of Computer Science and IT, and above all to our guide Dr. Kamalraj R, for guidance at the decisive points of this work and for being available whenever it mattered.

We are very thankful to our families and friends for their constant and unconditional encouragement and support.

NAME: Anush Kulal M
USN: 24MCAR0133

NAME: Akshita Bansal
USN: 24MCAR0131

NAME: Chandan Mathur
USN: 24MCAR0230

TABLE OF CONTENTS

Chapter	Title	Page No.
	Abstract	1
1	Introduction	3
	1.1 Overview	3
	1.2 Problem Background	3
	1.3 Motivation	4
	1.4 Objectives of the Project	4
	1.5 Scope of the Project	5
	1.6 Organization of the Report	6
2	Literature Study	7
	2.1 Introduction	7
	2.2 Review of Traditional Manual Systems	7
	2.3 Review of Existing Digital Rental Applications	7
	2.4 Payment Models on Two-Sided Platforms	8
	2.5 Review of Token-Based Session Management	9
	2.6 Gap Analysis	10
3	System Analysis	11
	3.1 Existing System	11
	3.2 Limitations of the Existing System	11
	3.3 Proposed System	12
	3.4 Advantages of the Proposed System	12
	3.5 Feasibility Study	13
4	System Design and Architecture	15
	4.1 System Architecture	15
	4.2 Client Architecture and the View-Model	16
	4.3 API and Authorisation Layer	16
	4.4 Payment and Settlement Design	17
	4.5 Session and Authentication Design	18
	4.6 Database Design	19
5	System Requirements	20
	5.1 Hardware Requirements	20
	5.2 Software Requirements	20
	5.3 Programming Platform Features	21
	5.4 Functional Requirements	21
	5.5 Non-Functional Requirements	22
6	Implementation	23

Chapter	Title	Page No.
	6.1 Authentication and Session Module	23
	6.2 Property, Room and Tenant Management Module	23
	6.3 Rent Cycle and Due Date Module	24
	6.4 Payment and Confirmation Module	24
	6.5 Payment Gateway Connection Module	25
	6.6 Notification and Reminder Module	25
	6.7 Maintenance Request Module	25
	6.8 Tenant Behaviour Score Module	26
	6.9 Tenant Portal Module	26
	6.10 Testing	26
7	Results and Screenshots	30
	7.1 Landlord Overview	30
	7.2 Tenant Roster	31
	7.3 Rooms and Properties	32
	7.4 Ledger	33
	7.5 Connecting a Payment Gateway	34
	7.6 Tenant Home	35
	7.7 Paying Rent	36
	7.8 Tenant Profile and Payment History	37
	7.9 Maintenance	38
8	Conclusion and Future Enhancement	40
	8.1 Conclusion	40
	8.2 Limitations of the Present Work	41
	8.3 Future Enhancement	41
	References	19

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of Reviewed Categories of Existing Systems	8
2.2	Structural Options for Moving Money on a Platform	9
3.1	Existing System Compared with the Proposed System	12
3.2	Feasibility Analysis Summary	14
4.1	Evolution of the Session Design	18
4.2	Principal Tables and Their Role	19
5.1	Hardware Requirements	20
5.2	Software Requirements	20
6.1	Test Suites and Assertion Counts	26
6.2	Defects Found by the End-to-End Runs	28

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	System architecture: client, API, data, supporting services, and the money path outside all of them	15
7.1	Landlord overview: collection against expectation, overdue total, occupancy, six-month revenue and the open-ticket queue	31
7.2	Tenant roster with counted filters, due-state chips and behaviour scores	32
7.3	Room list showing occupancy, rent and tenant assignment per unit	33
7.4	Ledger of recorded payments with method, date and tenant	34
7.5	The online payments sheet: what it costs, what the gateway will ask for, and the key fields	35
7.6	Tenant home: rent due with countdown, deposit held, tenure, open requests and the landlord's contact	36
7.7	The payment screen with a gateway connected: the online route above the divider and the UPI route beneath it	37
7.8	Tenant profile: tenure, rent share, deposit, the behaviour score with its factors, and the payment timeline	38
7.9	Tenant help screen listing maintenance requests with their status	39

ABSTRACT

Rental accommodation is one of the largest and least organised parts of the urban housing economy in India. Paying guest establishments, private hostels and small independently owned apartment blocks house a very large share of students and working people in cities, yet the businesses behind them are still run on paper. Tenant names sit in a register at the front desk, rent arrives by UPI transfer or in cash, and the record of a payment is a screenshot on somebody's phone. The tenant who has paid holds no receipt the landlord has acknowledged, and the landlord with six rooms has no dependable way of telling who is behind.

TenantPro is a two-sided rent-management system built to replace that arrangement. It consists of a React Native client running on Expo SDK 54 and a Node and Express API over MySQL, with a landlord application and a separate tenant portal served from one codebase. The system is deployed and in use. At the commit documented here it comes to 38,536 lines across 124 source files, exposes 87 API endpoints over 22 database tables, and presents 28 screens.

The decision that shapes the rest of the design is that money never passes through the platform. Rent moves straight from tenant to landlord, either by UPI into the landlord's own virtual payment address or through a payment gateway connected to the landlord's own merchant account. This keeps the operator outside the Reserve Bank of India's payment-aggregator perimeter, which carries an authorisation requirement and a net-worth floor in the tens of crores, and it means the platform bills nobody and holds nobody's funds. Because an application that does not touch the money also cannot see it, every payment is treated as a claim until a second party vouches for it: the landlord tapping Confirm, or the gateway answering a server-to-server query. Both routes settle through one function, so they cannot disagree about when a rent cycle has cleared.

The build is verified by 457 assertions across eight test suites. Of these, 168 run end-to-end against a live server and a real MySQL instance, and 71 drive the compiled application inside a browser engine. The end-to-end runs found five defects that unit tests had not, including a cross-account authorisation hole, and all five have been fixed. The report documents the architecture, the implemented feature set with interface evidence, the payment and session-security models, the verification methodology, and the work that has been identified but not built.

Keywords: Rent Management, Property Management System, Tenant Portal, Direct Payment Model, Payment Aggregator Regulation, UPI, Razorpay Payment Links, Refresh Token Rotation, React Native, Expo, Node.js, MySQL, End-to-End Testing.

CHAPTER 1 — INTRODUCTION

1.1 Overview

Rental accommodation forms one of the most important segments of the urban housing economy. Paying guest establishments, private hostels and small apartment blocks between two and twenty rooms house a very large share of the students and working population of every Indian city. Most of these properties are owned and run by individuals rather than by companies, and most of them are managed without any software at all.

The typical independent property is run out of a register. Tenant names are written in when somebody moves in and struck out when they leave. Rent is collected by UPI transfer or in cash, and the only record of it is a transaction screenshot that the tenant sends over WhatsApp. Deposits are remembered rather than recorded. Maintenance complaints arrive as phone calls and are resolved, or forgotten, with nothing written down either way. The arrangement works at one property with four rooms. It stops working at two properties with twelve.

TenantPro was built to replace that patchwork with a single record that both sides of the relationship can see. It is a two-sided system: the landlord gets a roster of properties, rooms and tenants, with due dates worked out from each tenancy's own anniversary, and the tenant gets a view of the same records from the other direction, showing what they owe, when it is due, what they have already paid, and which of it the landlord has acknowledged. The client is a React Native application distributed as an Android build; the server is a Node and Express API over MySQL, deployed and running.

1.2 Problem Background

The problems with manual property management are familiar to anybody who has run a rented property for more than a year. Rent amounts are recorded inconsistently or not at all. A payment made on the first of the month and a payment made on the nineteenth look identical in the register once both have been ticked off. When a tenant insists they paid and the landlord cannot find the entry, the dispute has no evidence on either side beyond a screenshot that proves a transfer happened but not which month it was meant for.

Deposits are the sharpest version of the same problem. A deposit is collected at the start of a tenancy that may run for two years, and it is settled at the end of it. In between, the only record is usually a figure in the owner's memory or a line in a notebook that has since been replaced. Disagreements at move-out are common and are rarely resolved in the tenant's favour, because the tenant has no record at all.

On the landlord's side, the absence of a consolidated view is what limits growth. An owner with a single property can hold its state in their head. An owner with three properties cannot, and the first thing they lose is a reliable answer to the simplest question in the business: who has not paid this month. Reminders then go out either to everybody, which annoys the tenants who have paid, or to nobody, which is why collection slips.

1.3 Motivation

The motivation for this project came from watching both sides of the relationship fail at the same task for different reasons. Owners described spending the first week of every month chasing payments they could not confirm. Tenants described paying on time and then being asked again a week later, with no way to prove the first payment had been received.

At the same time the surrounding environment has become far more favourable than it was even five years ago. UPI has made digital payment the default in India rather than the exception, which means the money movement itself is already solved and does not need to be reinvented. Cloud hosting is cheap and reliable. Practically every tenant carries a smartphone. What is missing is not a payment mechanism but a shared record of what the payment was for.

A review of the applications already in this space showed a consistent pattern. The simpler ones are little more than digital notebooks with no tenant-facing side at all, so the tenant remains in exactly the position they were in with the paper register. The more ambitious ones solve the record problem by routing the rent through their own account and settling it on to the landlord, which solves the visibility problem by creating a regulatory one. Neither approach fitted the situation this project set out to serve.

1.4 Objectives of the Project

The project was shaped around a set of defined objectives.

1. To replace manual record keeping with a structured digital record of properties, rooms, tenants, tenancies, rent, deposits and payments, held in a relational database rather than in a register.
2. To give the tenant a first-class view of the same record, so that the person paying the rent can see what is owed, what has been paid, and what the landlord has formally acknowledged.
3. To compute each tenancy's due date from its own start date rather than from a shared calendar month, so that a tenant who moved in on the nineteenth is due on the nineteenth.
4. To make digital payment the ordinary route, both through UPI directly into the landlord's own account and through a payment gateway connected to the landlord's own merchant account.
5. To keep the platform outside the flow of funds entirely, so that it neither holds nor transmits any tenant's money, and therefore does not fall within the regulatory perimeter that applies to payment aggregators.
6. To ensure that no payment is treated as settled on the strength of the payer's own assertion, by requiring confirmation from the landlord or from the gateway before a rent cycle is advanced.
7. To provide automated rent reminders across the channels a tenant actually uses, and a shared maintenance thread so that complaints are recorded rather than remembered.
8. To build a session and authentication design in which a session can actually be ended, so that a stolen phone, a departed tenant or a password change has a real effect.
9. To verify the resulting system by exercising every endpoint against a real database and the compiled application against a real browser engine, rather than relying on unit tests over mocked dependencies.

1.5 Scope of the Project

The scope of TenantPro covers the operational lifecycle of an independently owned rental property. This includes registering a landlord account, creating properties and rooms, onboarding tenants either by direct entry or by an invite code the tenant redeems, recording rent and deposit terms, generating due dates, issuing reminders, accepting and confirming payments, maintaining a searchable ledger, running maintenance request threads, holding identity documents, computing a payment-behaviour score visible to

both parties, recording notice periods and leave dates, and giving the tenant a portal in which all of the above is visible from their own side.

The scope deliberately excludes several activities that are sometimes bundled into property management platforms. The system does not hold or transmit money, and it does not act as an escrow. It does not draft, store or register rental agreements, and it makes no claim to address the legal layer of the tenancy. It is not a listing marketplace and performs no brokerage function. It does not provide accounting exports or tax filing. Formal expense and vendor management are outside the present scope. Finally, the client is an Android application; no iOS build has been produced.

1.6 Organization of the Report

The remainder of this report is organised into seven further chapters. Chapter 2 reviews the manual practices and the existing digital applications in this space, together with the regulatory and security literature that bears on the design, and ends with a gap analysis. Chapter 3 analyses the existing system, states its limitations, describes the proposed system and reports a feasibility study carried out along technical, economic, operational and regulatory axes. Chapter 4 presents the system architecture and the design of the payment, session and database layers. Chapter 5 states the hardware, software, functional and non-functional requirements. Chapter 6 describes the implementation module by module and reports the testing methodology and its results. Chapter 7 presents the finished interface with screenshots taken from the running application. Chapter 8 concludes, states the limitations of the present work honestly, and sets out the directions identified for future enhancement.

CHAPTER 2 — LITERATURE STUDY

2.1 Introduction

A project of this kind begins with a review of what already exists. For TenantPro this meant looking at three bodies of material: the informal manual practices that currently dominate the independent rental sector, the digital applications that have been built to replace them, and the regulatory and security literature that constrains how any platform in this space may be built. The third of these turned out to matter more than the second, because the central architectural decision in this project is a direct consequence of it.

2.2 Review of Traditional Manual Systems

There is a substantial body of management literature on why small businesses remain on manual systems even when digital alternatives are cheap and available. The recurring findings are that the perceived cost of switching is dominated by the time and risk of transition rather than by the price of the software, that the owner is usually also the operator and has no spare capacity for a migration, and that manual systems fail quietly rather than loudly, so their true cost is never measured.

Studies of small-scale hospitality and rental businesses consistently report that manual record keeping understates actual losses. Missed payments, partial payments that were never followed up, and deposits that were returned in full when deductions were due do not appear anywhere as losses, because no record exists against which to compare. The owner's impression of the business is therefore systematically more favourable than its accounts would be, which further weakens the case for change.

For this project, the relevant conclusion from that literature was not that owners are irrational but that any replacement has to be usable from the first day without a migration project, and has to pay for itself in a way the owner can see. That shaped a number of decisions later in the design, particularly the choice to let a landlord run the system with no payment gateway connected at all and still get the full benefit of the record.

2.3 Review of Existing Digital Rental Applications

A number of applications now target the rental property market in India. They range from very simple mobile apps built by independent developers, through mid-tier rent-collection

products, to full property management suites aimed at organised co-living operators. Reviewing them revealed a consistent split.

The simpler applications are, in effect, digital registers. They give the owner somewhere to type what they would otherwise have written down, which is a real improvement, but they have no tenant-facing side. The tenant's position is unchanged: they still have no acknowledged receipt and no view of their own history. Several of them also model the property as a flat list of tenants with no notion of a property containing rooms containing occupants, which makes them unusable above a single small property.

The more capable products do have a tenant side, and they solve the confirmation problem convincingly, because they collect the rent themselves. The tenant pays the platform, the platform marks the payment as received and then settles the money on to the landlord. This is clean from a software point of view and places the platform squarely inside the regulatory perimeter discussed in the next section. It also means the landlord's rent passes through somebody else's account on its way to them, which several of the owners consulted during this project objected to without prompting.

Table 2.1 — Comparison of Reviewed Categories of Existing Systems

Category	Tenant-facing view	Property hierarchy	Handles money	Suitability for independent owner
Paper register	None	Implicit, in the owner's head	No	Workable at one small property only
Simple digital notebook apps	None or read-only list	Flat tenant list	No	Breaks down beyond one property
Rent collection platforms	Yes	Partial	Yes, collects and settles	Regulated activity; owner loses direct settlement
Organised co-living suites	Yes	Full	Yes	Priced and scoped for operators, not individuals
TenantPro	Full parallel portal	Property to room to tenant	No, payment is direct	Designed for this segment

2.4 Payment Models on Two-Sided Platforms

Any platform that sits between a payer and a payee has three structural options, and the choice between them is a regulatory decision before it is a technical one.

In the aggregator model the platform receives the customer's money into its own account and then pays the merchant. Under the Reserve Bank of India's directions on payment aggregators, this is a regulated activity. An aggregator may not hold customer funds in its

ordinary account; the money must sit in a segregated escrow with a scheduled commercial bank, it may not earn interest for the platform, and it must be settled out within prescribed timelines. Obtaining the authorisation requires clearing a net-worth floor in the tens of crores. Ride-hailing platforms are the familiar example of this model.

In the split settlement model the platform does not hold the money itself, but it instructs a licensed gateway to split an incoming payment between a platform account and a linked sub-merchant account. The gateway holds the licence and performs the know-your-customer checks on the sub-merchants. Razorpay Route is the common implementation in India. This removes the onboarding burden from the platform and is attractive at scale, but funds still pass through a platform-controlled account, and the 2025 directions restrict how such arrangements may be structured.

In the direct model the platform never touches the money at all. The payer pays the payee, and the platform's role is limited to presenting the correct amount, the correct destination and a reference, and to recording what happened afterwards. Namma Yatri is the best-documented Indian precedent: a ride-hailing service in which the fare passes directly from rider to driver and the platform takes no commission, sustaining itself by other means. TenantPro occupies this third position.

Table 2.2 — Structural Options for Moving Money on a Platform

Model	Who holds the funds	Authorisation required	Example
Aggregator	The platform, in a ring-fenced escrow at a scheduled commercial bank	RBI payment aggregator licence	Uber, Ola
Split settlement	The gateway, settling to linked sub-merchant accounts	Held by the gateway, not the platform	Razorpay Route
Direct	Nobody; the payer pays the payee	None	Namma Yatri, TenantPro

2.5 Review of Token-Based Session Management

The security literature on session management for mobile clients is dominated by the trade-off between stateless and stateful tokens. A signed access token of the JSON Web Token family is verified by signature alone, with no database lookup, which is what makes it fast and what makes it scale. The cost of that property is that nothing can withdraw a token once issued: a stolen device, a tenancy that has ended or a password

that has been changed has no effect on a token already in circulation until it expires on its own.

The current best practice, set out in the OAuth 2.0 security guidance, is to pair a short-lived access token with a long-lived refresh token that is stored server-side, so that revocation has somewhere to act. Two further properties are recommended. The first is rotation: presenting a refresh token spends it and returns a replacement, so that a long-lived secret is never sent repeatedly across the network. The second is reuse detection: because rotation guarantees that the honest client always holds the newest token, the presentation of a token that has already been spent implies that a copy exists somewhere it should not, and the safe response is to end every session on that account.

The literature is also clear that the client half of this design is where it usually fails. A client that signs the user out on any authentication-looking error will sign them out on a network failure. A client that fires several requests at once on a cold start, all of which fail on the same expired token, will issue several refreshes, all but the first of which replay a spent token and are therefore read by a correctly implemented server as theft. Both failure modes were explicitly designed against in this project and are covered by tests.

2.6 Gap Analysis

Drawing the threads together, a clear gap emerges. Manual systems continue to dominate the independent rental sector despite being demonstrably lossy, because the switching cost is real and the losses are invisible. The simpler digital applications reduce the owner's effort but leave the tenant exactly where they were, which means the dispute that the system most needs to prevent is the one it does nothing about. The more capable platforms give the tenant a real view, but they achieve it by taking custody of the rent, which is a regulated activity and is in any case unattractive to the owners this project set out to serve.

What is missing is a system that gives both sides a shared, authoritative record of a tenancy while leaving the money exactly where it already flows, from the tenant's account into the landlord's. That is the gap TenantPro is designed to fill, and the confirmation model described in Chapter 4 is the mechanism by which it is possible to keep an accurate record of payments the platform is deliberately unable to observe.

CHAPTER 3 — SYSTEM ANALYSIS

3.1 Existing System

The existing system, as observed across the properties studied during this project, is a combination of a physical register at the property, a notebook or spreadsheet held by the owner, a WhatsApp thread per tenant, and the owner's memory. New tenants are entered in the register with a phone number and a room. Rent terms are agreed verbally or in a simple written agreement that is then filed and not referred to again. Payment is by UPI or cash. A UPI payment generates a confirmation in the tenant's own banking app, which the tenant screenshots and sends to the landlord; a cash payment generates nothing at all.

Reminders are sent by hand, usually in a burst at the start of the month, and usually to everybody, because working out who has already paid takes longer than sending the message. Maintenance complaints arrive by phone call or WhatsApp message and are tracked, if at all, in the same thread as everything else. Deposits are recorded once, at the start, and recalled at the end. Notice periods are agreed in conversation.

In the best case a diligent owner keeps this running with occasional errors. In the common case entries are missed, payments are double-chased, a deposit figure is disputed at move-out, and a maintenance complaint made three weeks ago has simply been forgotten by both parties.

3.2 Limitations of the Existing System

The limitations can be grouped under several heads. Accuracy is compromised at every step, because every figure is transcribed by hand at least once and some are never written down at all. Timeliness is poor, because the owner's view of who has paid lags reality by however long it takes them to reconcile screenshots against the register. Evidence is one-sided: the tenant holds proof that a transfer occurred but no proof of what it settled, and the landlord holds a register entry that the tenant has never seen.

Scalability fails sharply rather than gradually. The arrangement is workable at one property and unworkable at three, which is precisely the point at which an owner would otherwise expand. Accountability is absent, since nothing records who entered what or when. And the tenant, who is the paying customer of the business, is given no interface at all, which is the root of most of the disputes the system generates.

3.3 Proposed System

The proposed system replaces the register and the notebook with a relational record and gives both parties an application over it. Properties contain rooms; rooms contain tenancies; each tenancy carries its own rent, deposit, start date and split across occupants. Due dates are derived from the tenancy's own start date, so a tenant who moved in on the nineteenth is due on the nineteenth rather than on the first.

Payments are recorded against the due date they settle, captured at the moment the payment is declared. A payment enters the system as a claim and becomes real only when a second party vouches for it. Where the landlord has connected a payment gateway, the second party is the gateway itself, asked directly by the server using the landlord's own credentials. Where they have not, the second party is the landlord, who confirms or rejects the claim with a reason the tenant can read. Either way the same settlement function advances the tenancy.

The tenant portal is a parallel application rather than a reduced copy of the landlord's. It answers a different question and is reached through a separate account type with its own table and its own token role. Reminders are automated and go out across email, push, SMS and WhatsApp. Maintenance requests become threads shared between the two parties, with status changes recorded in sequence alongside what was said. A payment-behaviour score is computed by one piece of code and shown identically to both sides, so that neither is arguing with a number the other cannot see.

3.4 Advantages of the Proposed System

The proposed system improves on the existing arrangement along every dimension on which the latter fails. It is accurate, because figures are computed rather than transcribed. It is timely, because the landlord's view of collection is derived from the same rows the tenant's view is derived from. It is evidenced on both sides, because confirmation is an explicit, recorded act rather than an assumption. It scales, because a second and third property cost nothing but data entry. And it is accountable, because every payment, confirmation, rejection and status change is a row with a timestamp.

Table 3.1 — Existing System Compared with the Proposed System

Aspect	Existing system	Proposed system
Record of tenancy	Register entry and memory	Relational record, visible to both parties

Aspect	Existing system	Proposed system
Due date	Shared calendar month, applied loosely	Derived from each tenancy's own start date
Proof of payment	Tenant's screenshot of a transfer	Confirmed payment row carrying the due date it settled
Who confirms	Nobody; assumed on sight of a screenshot	Landlord, or the gateway asked server-to-server
Reminders	Sent by hand, usually to everybody	Automated daily job, addressed to those actually due
Deposit record	Remembered	Stored, and shown on the tenant's own screen
Maintenance	Phone call, tracked informally	Shared thread with recorded status transitions
Tenant's view	None	Full portal: dues, history, receipts, score, documents
Scalability	Fails at two or three properties	Unaffected by portfolio size

3.5 Feasibility Study

A feasibility study was carried out along four axes, the fourth of which is not standard but was decisive for this project.

3.5.1 Technical Feasibility

All the building blocks required are mature. React Native with Expo provides a single codebase targeting Android with over-the-air updates for JavaScript changes. Node and Express over MySQL is an unremarkable and well-documented server stack. Payment links, transactional email, push notification and messaging are all available as documented HTTPS APIs. Nothing in the design requires a technology that is experimental or that the project team would have to build from first principles. One technical constraint did shape the design materially and is discussed in Chapter 4: native modules cannot be delivered over the air, so any feature requiring a native software development kit would freeze the release cadence of everything around it.

3.5.2 Economic Feasibility

The running costs are modest. Cloud hosting for the API and the database, transactional email, push delivery and media storage fall within free or low-cost tiers at the scale of a few properties, and scale gracefully beyond it. Because the platform does not process payments, it carries no payment-processing cost of its own. Where a landlord chooses to connect a payment gateway, the gateway's fee is charged by the gateway to that landlord

against their own merchant account; the platform is not a party to it and bills nothing. The one identified recurring cost in the roadmap is a verified sending domain for transactional mail, at roughly one thousand rupees a year.

3.5.3 Operational Feasibility

Operational feasibility was examined most carefully, because the target users are not necessarily comfortable with software. Two design consequences followed. First, the system has to be fully useful with no payment gateway connected, because gateway onboarding involves a third party's know-your-customer process that takes days; a landlord who cannot use the product until that completes will not come back. Second, every screen that asks for a commitment states its cost and its prerequisites before the user begins, since a landlord who expects a five-minute task and meets a week-long one concludes the application is broken rather than that the gateway is slow.

3.5.4 Regulatory Feasibility

The regulatory axis determined the architecture. Collecting rent centrally and paying it out is payment aggregation under the Reserve Bank of India's directions, with the escrow obligations and net-worth floor described in Chapter 2. A student project cannot satisfy those conditions, and neither can the individual landlords the system serves. The system was therefore designed so that the question does not arise: the money moves directly from the tenant to the landlord, by UPI or through the landlord's own merchant account, and the platform neither receives it nor instructs its disbursement.

Table 3.2 — Feasibility Analysis Summary

Axis	Finding	Consequence for the design
Technical	All components mature and documented	Native modules avoided so that updates ship over the air
Economic	Low fixed cost; no payment-processing cost borne by the platform	Platform bills nobody; gateway fees are the landlord's own
Operational	Users are not necessarily technical; onboarding friction is fatal	System fully usable with no gateway connected
Regulatory	Central collection would be payment aggregation	Direct tenant-to-landlord payment; platform outside the flow of funds

CHAPTER 4 — SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

TenantPro follows a three-tier design. The presentation tier is a React Native client that serves both the landlord application and the tenant portal from one codebase. The application tier is a Node and Express API organised as routers, each with a role guard mounted once at the top. The data tier is MySQL, holding 22 tables that cover accounts, properties, rooms, tenancies, payments, maintenance threads, documents, notifications and sessions.

Around these three tiers sit the supporting services: a transactional mail provider, push notification delivery, media storage for identity documents and maintenance photographs, and the payment gateway. The last of these is drawn differently in the figure below for a reason. Every piece of information about a tenancy passes through the server. No part of the payment does. The dashed band in the figure is the money path; it touches the tenant and the landlord, and nothing in between.

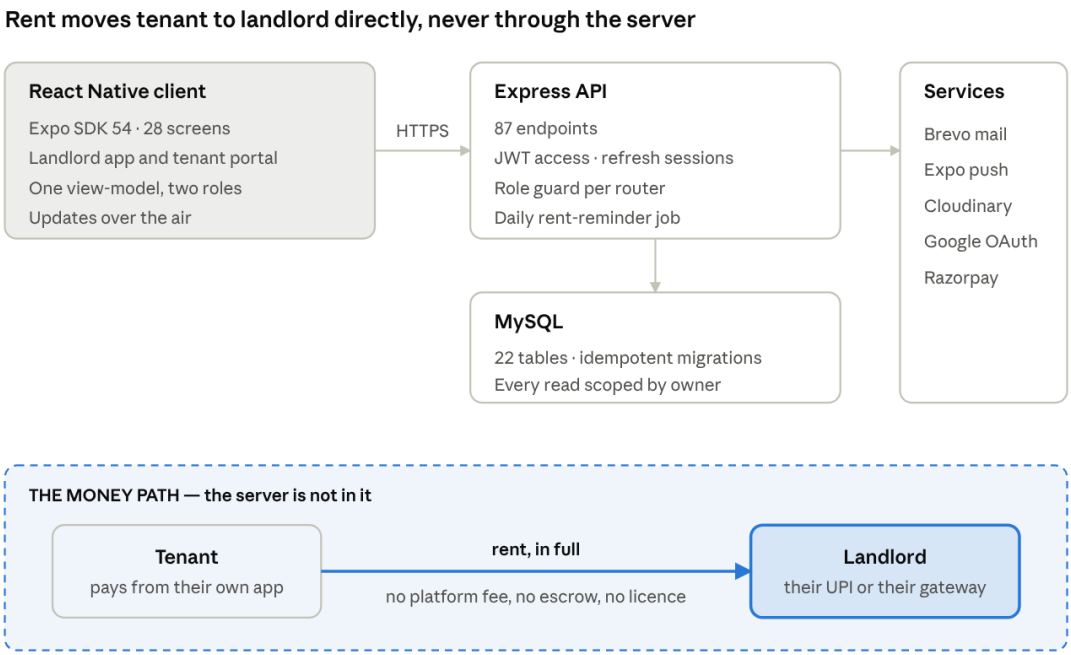


Fig 4.1 — System architecture: client, API, data, supporting services, and the money path outside all of them

4.2 Client Architecture and the View-Model

The client is a single React Native codebase serving both roles. Rather than scattering presentation logic across screens, the application derives everything the screens render through one pure function, which maps the current state and the server payload to a view-model. A screen therefore contains layout and very little else.

This has two consequences worth stating. The first is testability: because the function is pure and has no dependency on a device, the behaviour of the interface can be asserted directly, and one of the test suites described in Chapter 6 does exactly that. The second is consistency: two screens showing the same fact cannot drift apart, because both are reading the same derived value rather than each computing its own.

A constraint from the delivery mechanism shaped the feature set. Updates reach the installed client through the Expo updates mechanism, which can replace JavaScript but cannot replace native modules. Any feature that depends on a native software development kit can only be delivered through a full store release, and would hold back every other change queued behind it. Features of that kind were therefore declined wherever a design existed that ships like any other change. The payment integration is the clearest example, and is described in section 4.4.

4.3 API and Authorisation Layer

The API is organised as routers, each with its role guard mounted once at the top rather than repeated in every handler. Landlord and tenant accounts live in separate tables with their own auto-increments, which means that knowing who somebody is does not by itself settle what they may do: a tenant token presented on a landlord route is rejected by the guard, and tenant handlers scope every query through the link from the tenant account to the tenancy record.

Ownership checks are a second, separate layer. Every read is scoped by owner, but writes that accept an identifier from the request body need an explicit check that the referenced row belongs to the caller. Those checks are centralised in a single helper module rather than written inline at each site, for a reason recorded in Chapter 6: when they were written inline, one of three endpoints was missed.

Schema changes apply themselves at boot and apply safely more than once. The schema file runs its table creation statements conditionally on every start, and column-level migrations run behind existence guards. No deploy has required a manual migration step.

4.4 Payment and Settlement Design

Two payment routes exist, and both are direct.

The first is UPI. The tenant's screen carries the landlord's own virtual payment address, the exact amount and a generated reference, encoded both as a scannable code and as an intent that hands off to whichever UPI application the tenant already uses. The money moves from the tenant's bank to the landlord's. It costs both parties nothing and it costs the platform nothing.

The second is a payment gateway. A landlord may connect their own Razorpay merchant account by entering their key and secret, which are verified against the gateway before being stored and are then sealed with authenticated encryption and never returned to any client. When a tenant pays by this route the server creates a payment link against that landlord's own credentials; the tenant pays on the gateway's hosted page and the money settles to that landlord. The gateway charges its fee to the landlord through the landlord's own account. The platform bills nothing, because it is party to nothing.

Payment links were chosen in preference to the gateway's React Native software development kit for the reason given in section 4.2: the kit is a native module, whereas a link is plain REST over HTTPS, so the payment feature ships over the air like every other change.

The confirmation model binds the two routes together. A payment enters the system as a declared row, which is a claim and clears nothing. It becomes real when a second party vouches for it, either the landlord tapping Confirm or the gateway being asked directly, server-to-server, using that landlord's credentials, and answering that the link was paid. Both paths end in the same settlement function, which flips the status and advances the tenancy's next due date inside one transaction. Written twice, the two copies would eventually disagree about when a month had cleared; it is therefore written once.

The gateway is polled rather than trusted to call back. A webhook would require every landlord to configure a URL and a shared secret in a dashboard, and a feature gated on

that step is one most landlords would never finish switching on. The signed webhook path exists and remains supported, but it is the secondary route.

4.5 Session and Authentication Design

The session design went through three versions, and the sequence is worth recording because each of the first two fixed something real and created the next problem in doing so.

Table 4.1 — Evolution of the Session Design

Stage	Access token	Can be revoked	Failure it solved	Failure it left
Original	7 days, fixed	No	—	Weekly logout for every user
Sliding	30 days, renewed on use	No	The weekly logout	Nothing could be cancelled
Current	2 hours plus a refresh token	Yes	Revocation	Lag of up to two hours on the access path

Raising the lifetime from seven days to thirty did not stop people being logged out; it made it happen monthly. A fixed lifetime counts from the last sign-in, which says nothing about whether somebody is still using the application. Opening the app every morning earned the same ejection on day thirty as not opening it since February. The server therefore re-issues the access token on any authenticated request once it is past half its life, returning it on a response header that the client adopts, and reading the new token's lifetime from the old one's own issue and expiry claims rather than from a constant, so that a guest session keeps its own length.

A session is now a database row as well as a signature. The row holds a refresh token stored as a SHA-256 hash rather than as the token itself, since the only operation ever performed on it is comparison. Three properties follow. Rotation: spending a refresh token returns a replacement, so a sixty-day secret is never sent repeatedly across the network. Reuse detection: because the honest client always holds the newest token, a replay of a spent one implies a copy exists elsewhere, and every session on the account is revoked. Revocation on password reset: changing a password now ends every other session, which is what users had always assumed it did.

Revocation is immediate on the refresh path and lags by at most the access token's two-hour life on the access path. Closing that gap completely would require a database read on

every authenticated request, a cost almost no system pays. Two hours where it used to be thirty days is the improvement.

4.6 Database Design

The schema holds 22 tables and mirrors the entity structure closely. Landlord accounts and tenant accounts are separate tables, which is what allows identity and authorisation to be distinguished at the API layer. Properties belong to landlords, rooms belong to properties, and tenancies connect a tenant record to a room with their own rent, deposit, split and dates. Payments reference the tenancy and carry the due date they settled, which is the field that makes the question "was this on time?" answerable after the fact. Without it the answer is lost, because confirming a payment moves the next due date forward and overwrites the only evidence; its earlier absence is why the behaviour score once reported that no tenant had ever missed a payment.

Table 4.2 — Principal Tables and Their Role

Table group	Holds	Notes
Accounts	Landlord accounts, tenant accounts	Separate tables with independent keys
Portfolio	Properties, rooms	Rooms carry rent, capacity and notification preferences
Tenancy	Tenant records, assignments, notice periods	Due dates derived from each tenancy's own start date
Payments	Declared, confirmed and rejected payments	Each row stores the due date it settled
Maintenance	Requests and thread messages	Status transitions stored as events in the thread
Documents	Identity proofs and maintenance photographs	Stored externally; previews served under short-lived tokens
Notifications	Alerts, delivery records, seen markers	Seen marker is stamped by the server, not the device
Sessions	Refresh token hashes, expiry, revocation reason	Hash only; the token itself is never stored
Gateway	Per-landlord sealed gateway credentials	Encrypted with AES-256-GCM; never returned to a client

CHAPTER 5 — SYSTEM REQUIREMENTS

5.1 Hardware Requirements

TenantPro is a client-server system. For the end user the requirements are minimal: an Android smartphone of recent vintage with an internet connection. The server-side requirements are those of a small Node process and a MySQL instance, both of which run comfortably on entry-level cloud infrastructure at the scale this system targets.

Table 5.1 — Hardware Requirements

Component	Minimum	Recommended
End-user device	Android 8.0 smartphone, 2 GB RAM	Android 11 or later, 4 GB RAM
Network	2 Mbps mobile data	4G or Wi-Fi
Application server	1 vCPU, 512 MB RAM	2 vCPU, 2 GB RAM
Database server	1 vCPU, 1 GB RAM, 10 GB storage	2 vCPU, 2 GB RAM, 25 GB storage
Development machine	4 GB RAM, 20 GB free disk	8 GB RAM or more, SSD

5.2 Software Requirements

The software stack was chosen to balance developer productivity, delivery flexibility and cost.

Table 5.2 — Software Requirements

Layer	Technology	Purpose
Client framework	React Native on Expo SDK 54	Single codebase for both roles, Android build
Client delivery	Expo updates and EAS Build	Over-the-air JavaScript updates; store builds when native changes
Server runtime	Node.js with Express	REST API, 87 endpoints across role-guarded routers
Database	MySQL	22 tables; schema applied idempotently at boot
Authentication	Signed access tokens with server-side refresh tokens	Short-lived access, revocable sessions
Encryption	AES-256-GCM, SHA-256	Sealed gateway credentials; hashed refresh tokens
Payments	Razorpay Payment Links over REST; UPI intent and QR	Direct settlement to the landlord
Messaging	Transactional email, push notifications, SMS and WhatsApp	Rent reminders and alerts

Layer	Technology	Purpose
Media	Cloud media storage	Identity documents and maintenance photographs
Testing	Plain Node scripts; Playwright with Chromium	Unit, integration, end-to-end and browser-driven suites
Version control	Git and GitHub, with GitHub Actions	Source control and build or update workflows

5.3 Programming Platform Features

React Native with Expo was chosen because it produces a single JavaScript codebase that runs on the device while still allowing most changes to be shipped without a store release. That last property is not a convenience but a constraint that shaped the feature set, as described in Chapter 4.

Node with Express was chosen for the server because using JavaScript on both sides removes a translation layer between client and server models, and because the Express router model maps cleanly onto the role-guard design. MySQL was chosen over a document store because the data is strongly relational: a payment is meaningless except in relation to a tenancy, which is meaningless except in relation to a room and a property.

All external integrations are reached through documented REST APIs with explicit error handling. One point of care is worth noting here and is revisited in Chapter 6: a request that fails to reach a third party is a different event from a request that reached it and was refused, and the two must not be reported to the user in the same words.

5.4 Functional Requirements

The system shall allow a landlord to register, sign in, and recover a forgotten password. It shall allow a landlord to create properties and rooms, with per-room rent, capacity, occupancy status and notification preferences. It shall allow tenants to be created directly or to join by redeeming an invite code, with the landlord accepting or declining the request and optionally assigning a room at the same time.

It shall derive each tenancy's due date from its own start date and maintain the due state of every tenant. It shall allow a tenant to declare a payment, and shall record that payment as a claim carrying the due date it is intended to settle. It shall allow a landlord to confirm or reject a claim, with a rejection reason visible to the tenant. Where a landlord has connected a gateway, it shall create a payment link against that landlord's credentials and

shall confirm the payment by querying the gateway directly rather than by trusting the client.

It shall maintain a searchable ledger of confirmed payments, scopable to a single property. It shall send automated rent reminders daily across the configured channels. It shall provide maintenance request threads shared between landlord and tenant, with status transitions recorded in sequence. It shall accept identity documents and serve previews under short-lived tokens. It shall compute a payment-behaviour score from the same code for both parties and itemise its factors. It shall record notice periods and compute leave dates server-side. It shall allow a tenant to view their dues, history, receipts, deposit, tenure and score in a dedicated portal.

5.5 Non-Functional Requirements

Usability comes first, because the users are not necessarily technical. Screens are expected to state the cost and the prerequisites of any action before the user commits to it, and error messages are expected to distinguish between a failure of the network, a refusal by a third party and a mistake by the user.

Security requires that credentials for third-party services be stored encrypted and never returned to a client, that refresh tokens be stored only as hashes, that sessions be revocable, and that authorisation be enforced on the server for every write that accepts an identifier from the client. Correctness requires that a payment never be treated as settled on the payer's assertion alone, and that status changes and due-date advances occur inside a single transaction.

Reliability requires that a transient network failure never sign a user out and never be reported as a rejected credential. Maintainability requires that behaviour shared between the two applications be implemented once. Portability requires that schema changes apply themselves at boot so that a deploy is a code deploy and nothing else.

CHAPTER 6 — IMPLEMENTATION

The implementation was carried out module by module, each module being a functional area with its own routes on the server and its own derived state on the client. The modules are described below in the order in which a landlord and a tenant encounter them.

6.1 Authentication and Session Module

This module registers accounts, authenticates them, issues sessions and ends them. Landlord and tenant accounts are held in separate tables, and the token issued to each carries a role that the API's guards act on. Sign-in may also be performed through a federated Google identity.

A session consists of a short-lived access token and a refresh token held as a hash in a sessions table. The access token is renewed transparently on any authenticated request once it passes half its life. The refresh token rotates on every use, and the presentation of a token that has already been spent revokes every session on the account. Signing out revokes the row with a recorded reason, and a password reset revokes every other session.

The client half received particular attention. A failed request triggers at most one refresh and one replay, and only a refusal from the refresh endpoint itself ends the session; a server fault, a timeout or a dead connection does not. All parallel refreshes share a single in-flight promise, so that an application firing eight requests on a cold start issues one refresh rather than eight. Without that, seven of the eight would replay a token the first had already spent, the server would correctly read the replay as theft, and the feature built to prevent lockouts would have been causing them.

6.2 Property, Room and Tenant Management Module

This module is where the landlord describes the shape of the business. Properties are created with a name, type and address; rooms are created within them with a number, rent, capacity and their own notification preferences. A room's occupancy status follows its occupants automatically.

Tenants may be created directly by the landlord or may join by redeeming an invite code, in which case the landlord sees the request with the tenant's details and any identity documents before accepting. A tenant may exist without a room; that state is reachable

from the roster rather than hidden, because a tenant who has been added but not yet placed is an ordinary situation rather than an error.

Every write in this module that accepts a room identifier from the request body checks that the room belongs to the caller. That check is centralised in one helper module. It was not always so, and the consequence is reported in section 6.10.

6.3 Rent Cycle and Due Date Module

Each tenancy carries its own start date, rent and split. The due date is computed from the tenancy's own anniversary rather than from a shared calendar month, which is what allows a tenant who moved in on the nineteenth to be due on the nineteenth. The module maintains the due state of every tenant and exposes it as the chip shown on the roster: due within a number of days, a number of days late, or a leaving date where notice has been given.

Advancing the due date is not performed by this module directly. It happens only inside the settlement transaction described in section 6.4, so that a due date can never move without a payment having settled, and a payment can never settle without the due date moving.

6.4 Payment and Confirmation Module

A payment is created as a declared row carrying the amount, a generated reference, the method, and the due date it is intended to settle. In that state it clears nothing; it is a claim.

Confirmation has two sources. The landlord may confirm or reject the claim from a queue that shows the full context needed to decide, and a rejection carries a reason that the tenant reads. Alternatively, where the payment was made through a connected gateway, the server queries the gateway directly using that landlord's credentials and settles the payment on the gateway's own answer. Both sources call the same settlement function, which flips the status and advances the tenancy's next due date inside one transaction.

On the tenant's side the distinction is made visible rather than hidden. A claim still awaiting the landlord is shown in amber as awaiting confirmation, not in the green reserved for money that has arrived, because showing the two alike is how a tenant comes to believe their month is clear when it is not.

6.5 Payment Gateway Connection Module

This module lets a landlord connect their own Razorpay merchant account. The sheet that does so states the gateway's fee next to the proposition, because describing one without the other is how somebody switches on a percentage fee without noticing. It also states what the gateway will require before the landlord sets off — a permanent account number, a bank account, one business proof, and realistically two to five working days — and notes that test keys work immediately and that tenants can carry on paying by UPI throughout.

Credentials are verified against the gateway before anything is stored, so that a mistyped secret is caught while the landlord still has the form in front of them. What is stored is sealed with AES-256-GCM and is never returned to any client. A failure to reach the gateway is reported as a network failure and not as a rejected credential; the word "rejected" is reserved for the gateway actually saying no.

6.6 Notification and Reminder Module

A scheduled job runs each morning and issues rent reminders to the tenants who are actually due, across email, push, SMS and WhatsApp according to the room's configured preferences, with every channel phrasing the same day identically. Beyond reminders, the module raises alerts for a landlord's reply on a maintenance thread, a payment confirmed or rejected, and the outcome of a join request.

The alert indicator derives its state from the same payload the screen renders rather than from a separate unread counter that could drift away from it. A server-stamped seen marker distinguishes an alert that is news from one that is merely still true; the server stamps it so that a device with the wrong clock cannot silence its own alerts.

6.7 Maintenance Request Module

A tenant raises a request with a category, a priority and a photograph if they have one, and then follows it in a thread shared with the landlord. Status changes land in that thread as events, in sequence with whatever was said around them, so that the history of a repair reads as one narrative rather than as a conversation plus a separate status field.

6.8 Tenant Behaviour Score Module

The module computes a payment-behaviour score from the tenant's record of on-time, late and missed payments. The score is produced by one piece of code and shown identically on both sides, which is the point of it: a tenant looking at a different number from the one their landlord sees cannot argue with either. The factors are itemised rather than reduced to a single dial, and where there is too little history to say anything the screen says so, instead of parking a needle at zero which would read as a verdict the tenant had earned.

6.9 Tenant Portal Module

The tenant portal is a parallel application rather than a reduced copy of the landlord's. Its home screen carries the rent card with the amount, the property and room, the exact due date and a countdown, and beneath it the deposit held and the tenure, both of which are facts tenants routinely cannot recall and have no other record of.

The module also covers guest accounts for tenants without an email address, identified by a code with a stay window enforced on the server; the conversion of a guest into a full account; joining a property by scanning or typing an invite code; receipts with status, reference and any rejection reason; giving and withdrawing notice with the real dates computed and shown before anything is written; uploading and removing identity documents; and directions to the property where the landlord has pinned it.

6.10 Testing

The build is verified by 457 assertions across eight suites. None of them uses a test framework; each suite is a plain Node script that exits non-zero when something fails, which keeps the test code free of dependencies and makes a failing run reproducible from a single command.

Table 6.1 — Test Suites and Assertion Counts

Suite	What it drives	Isolation	Assertions
secretbox	AES-256-GCM sealing of gateway credentials	Pure	24
session	Token renewal, through the real middleware and interceptor	Stubbed database	41
gateway	Gateway controllers	Stubbed database and network	72
vm.gateway	The real view-model function, native layer stubbed	Stubbed native	52

Suite	What it drives	Isolation	Assertions
refresh.client	The real request interceptor: refresh, retry, and when not to sign out	Stub adapter	29
refresh.e2e	Refresh tokens, reading the database directly	Live server and MySQL	42
api.e2e	All 87 endpoints	Live server and MySQL	126
ui.e2e	The compiled application in a browser engine	Live stack and Chromium	71
Total	—	—	457

6.10.1 Unit Testing

Unit tests cover the calculation-heavy and security-critical functions in isolation: the sealing and unsealing of gateway credentials, the hashing and comparison of refresh tokens, the renewal arithmetic that reads a token's remaining lifetime from its own claims, and the view-model function that maps server state to what the screens render.

6.10.2 Integration Testing

Integration tests exercise the controllers with their dependencies stubbed at the boundary rather than inside. The gateway suite, for example, runs the real controller code against a stubbed network, which is what allows the distinction between a refused credential and an unreachable host to be asserted directly rather than inferred.

6.10.3 End-to-End Testing

The end-to-end suites run against a live server and a real MySQL instance seeded by a deterministic generator. That generator deliberately produces the rows that break screens and never turn up when one is looking for them: a tenant with no room, a guest with no email, a guest whose stay has already ended, rent overdue and due today and upcoming and never set at all, a payment declared and undecided, another rejected with a reason, maintenance threads where the landlord spoke last and others where the tenant did, a tenant who has given notice, and a property with no rooms in it. The pseudo-random number generator is seeded, so any failing run can be reproduced exactly.

The refresh suite goes further and reads the sessions table directly, so that claims about rotation, reuse detection and revocation are checked against stored rows rather than against the API's own account of itself. A session was driven through three access-token expiries during a live browser run without the user being signed out, and signing out

through the interface was then confirmed to have written a revocation reason to the row and to have made the token unusable.

6.10.4 Browser-Driven User Interface Testing

The user interface suite runs the same JavaScript the phone runs — the same view-model, screens, request client and interceptors — laid out for the browser by the web renderer that an Expo web export produces, and driven through Chromium. What it cannot reach is anything behind a native module, so the camera, the image picker, push delivery and the UPI intent hand-off are out of its scope. For those it prints a skip rather than a pass, so that a green run never claims coverage it does not have.

Running the application in a desktop browser is not a substitute for a device, and it is not claimed as one here. It is, however, the only automated way available in this project's environment to assert that a real screen renders a real row correctly, and it found defects that no unit test had.

6.10.5 Defects Found

Driving every endpoint against a real database, and the application against a real browser, surfaced five defects that the unit tests had missed.

Table 6.2 — Defects Found by the End-to-End Runs

Defect	Severity	Status
Two of three endpoints accepting a room identifier checked ownership; the third did not, allowing one landlord's tenant to be filed into another landlord's room	Cross-account write	Fixed; the check is now one shared helper
A wrong password rendered "Your session expired", overriding the server's own message	User-facing	Fixed
Five required-field omissions produced HTTP 500 with a raw database error instead of a 400	Robustness	Fixed
Renaming a property without its address crashed on a NOT NULL column coerced on insert but not on update	Robustness	Fixed
A network failure reaching the gateway was reported as a rejected credential	User-facing	Fixed; transport failures now distinguished

The first of these is worth dwelling on, because it is the reason ownership checks were centralised. Every read in the system is scoped by owner, so the second landlord never saw the intruding row; the leak was that the first landlord's own tenant list then displayed a stranger's property name and room number. The defect was found by filing one

landlord's tenant into another's room through the API and receiving a cheerful success response.

Two further defects were found in the test harnesses themselves, which is worth recording. The first browser run never got past the permissions prompt, and because its assertions were phrased as the absence of a string that the prompt also happened to lack, half the suite passed while testing nothing at all. Separately, the failure reporter crashed on a failure whose context was undefined and took down the run it was supposed to be reporting. The lessons are to assert on what should be present rather than on what should be absent, and to keep the failure reporter the dullest code in the file.

CHAPTER 7 — RESULTS AND SCREENSHOTS

The system meets the functional and non-functional requirements set out in Chapter 5. A landlord can manage several properties and their rooms, track every tenant's dues, confirm or reject payments, run maintenance threads and connect a payment gateway. A tenant can see what they owe, pay it by either route, follow their own history, raise maintenance requests and give notice. The screenshots in this chapter are of the running application against a seeded database of three properties, six rooms and ten tenants. No figure in them is placeholder text.

7.1 Landlord Overview

The landing screen answers the two questions that make a landlord open the application at all: who has paid, and who has not. Collection is shown against what was expected rather than as a bare total, because a number with no denominator is not something anyone can act on. Overdue is given a card of its own with a count of people attached. The revenue series is built from confirmed payments only; claims that have been declared but not yet confirmed are excluded on purpose, since including them would inflate income on nothing more than a tenant's word. The bell carries a live count of whatever is waiting on a decision.

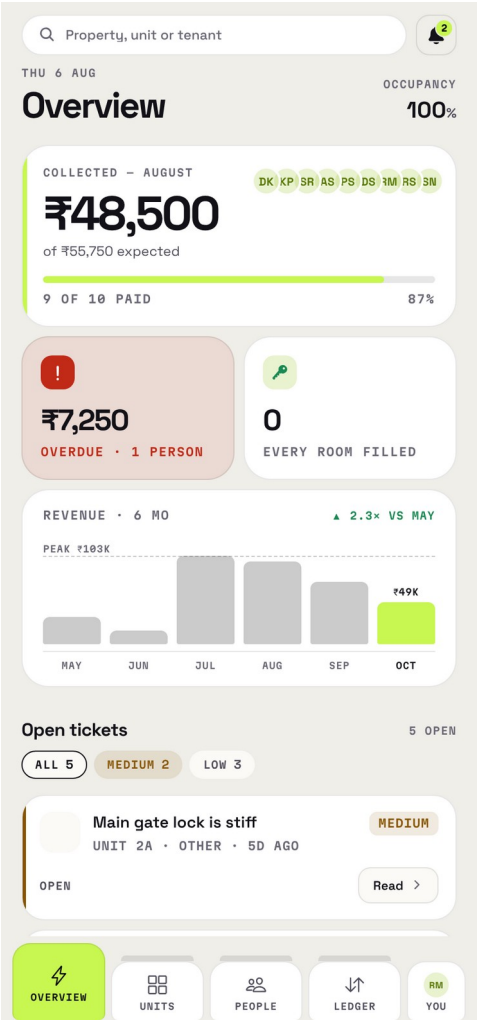


Fig 7.1 — Landlord overview: collection against expectation, overdue total, occupancy, six-month revenue and the open-ticket queue

7.2 Tenant Roster

Each row carries the tenant's property and room, their share of the rent, a payment-behaviour score and a due-state chip. The filter chips carry counts, so the distribution is visible before any of them is tapped. The unassigned filter is not an afterthought: a tenant is allowed to exist without a room, and the end-to-end suite checks that such a row renders without a broken room label.

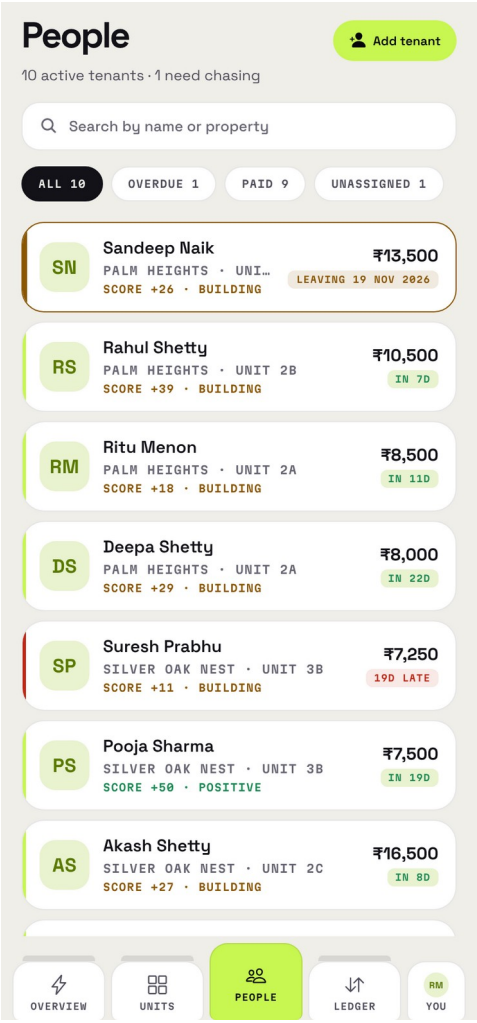


Fig 7.2 — Tenant roster with counted filters, due-state chips and behaviour scores

7.3 Rooms and Properties

Properties hold rooms and rooms hold tenants, each room with its own rent, capacity, and a split across occupants that can be equal or set by hand. A room's status follows its occupancy on its own. Notification preferences are per room, with each channel switched independently, because a landlord rarely wants the same channel across a whole portfolio.

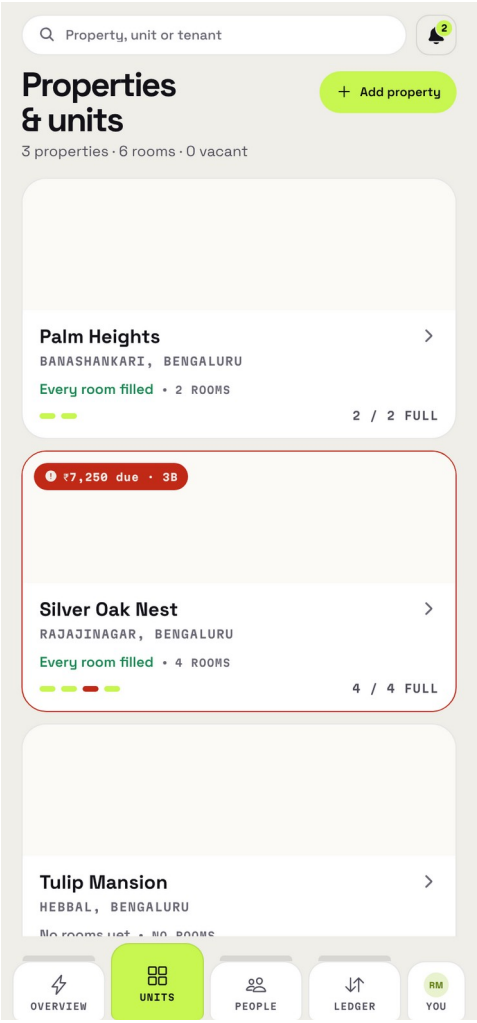


Fig 7.3 — Room list showing occupancy, rent and tenant assignment per unit

7.4 Ledger

The ledger holds every confirmed payment, searchable and narrowable to a single property. Each payment stores the due date it settled, captured at the moment of declaration, which is the field that makes punctuality answerable after the fact.

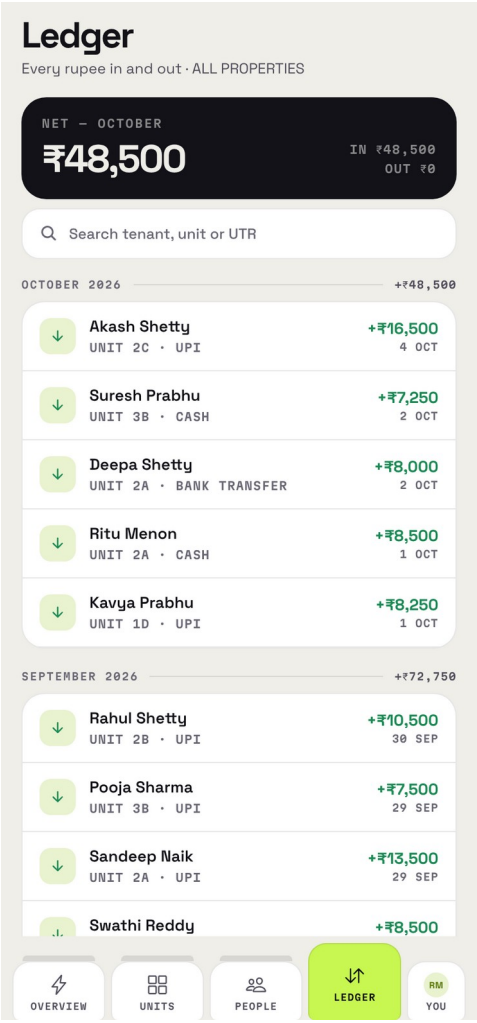


Fig 7.4 — Ledger of recorded payments with method, date and tenant

7.5 Connecting a Payment Gateway

The sheet states the fee next to the offer, and sets out what the gateway will ask for and how long it will take before the landlord begins. Credentials are verified against the gateway before being stored and are sealed afterwards.

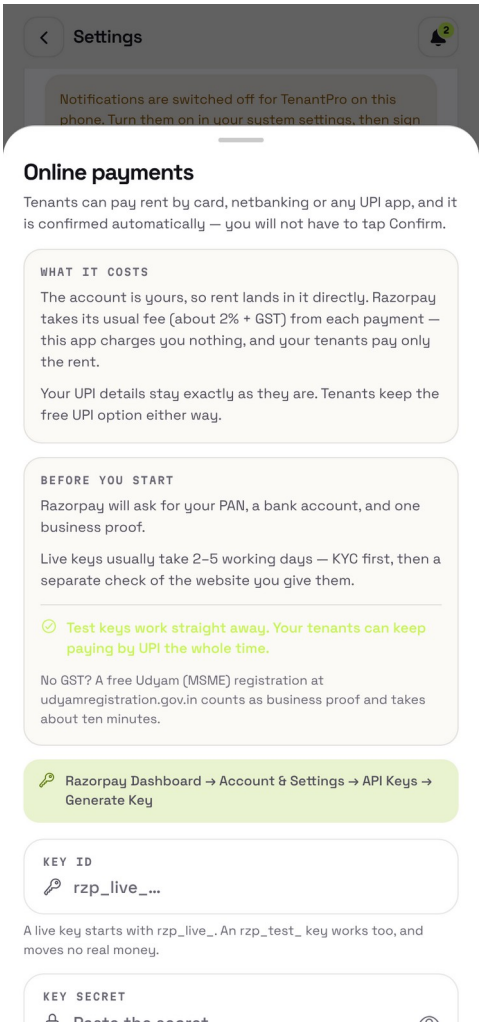


Fig 7.5 — The online payments sheet: what it costs, what the gateway will ask for, and the key fields

7.6 Tenant Home

The rent card carries the amount, the property and room, the exact due date and a countdown to it. Deposit held and tenure sit underneath, and the landlord's contact and any open requests follow. The alert indicator works off the same payload the screen renders rather than a separate counter.

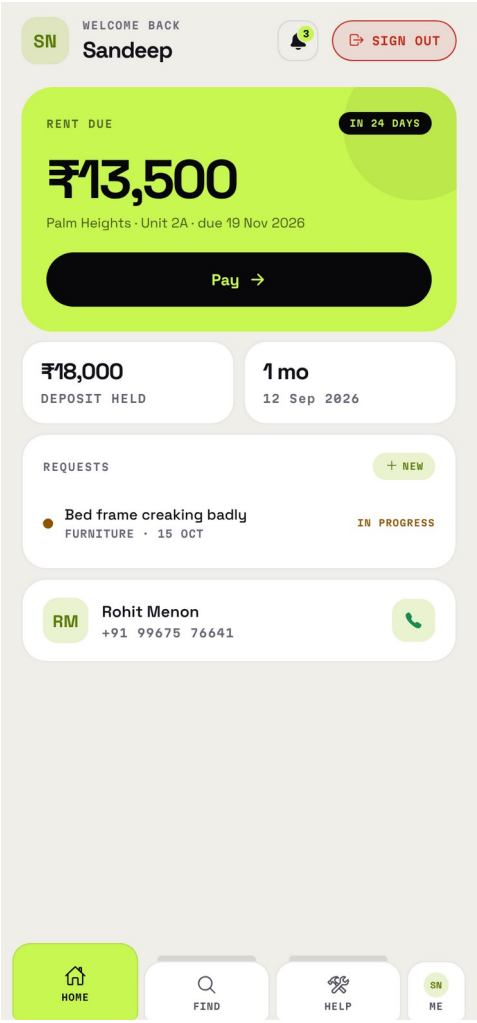


Fig 7.6 — Tenant home: rent due with countdown, deposit held, tenure, open requests and the landlord's contact

7.7 Paying Rent

The screen shows what is owed, a reference, and zero as the platform fee, which is literally true on both routes. Where the landlord has connected a gateway, the card and netbanking route appears first because it is the only one that confirms itself; beneath a divider sits the UPI route, with the landlord's own address, the amount and the reference encoded for scanning from another phone and as a hand-off to a UPI application on this one. The two buttons are styled differently on purpose, since two filled primary buttons amount to two primary actions, which is the same as having none. After returning from a gateway page the button reads "Check my payment" rather than "Yes, I paid", because on that route the answer is not the tenant's to give.

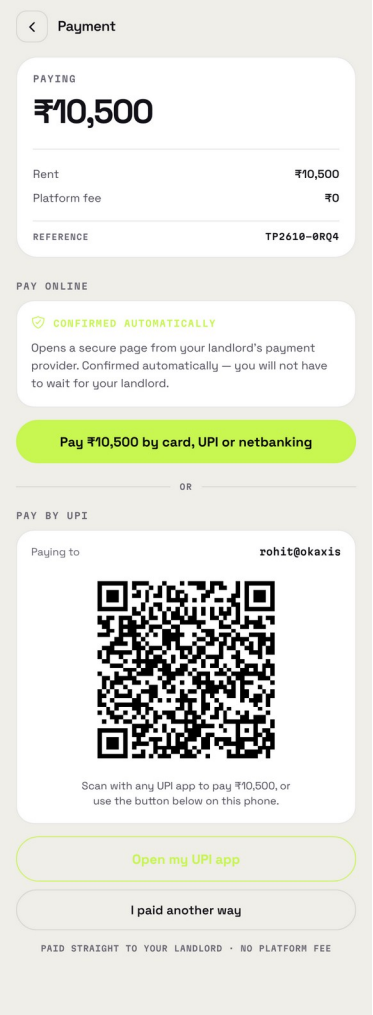


Fig 7.7 — The payment screen with a gateway connected: the online route above the divider and the UPI route beneath it

7.8 Tenant Profile and Payment History

The profile consolidates tenure, rent share, deposit and the behaviour score with its factors itemised, followed by the payment timeline. The timeline is built from real rows, and a claim still waiting on the landlord shows amber rather than the green reserved for money that has arrived.

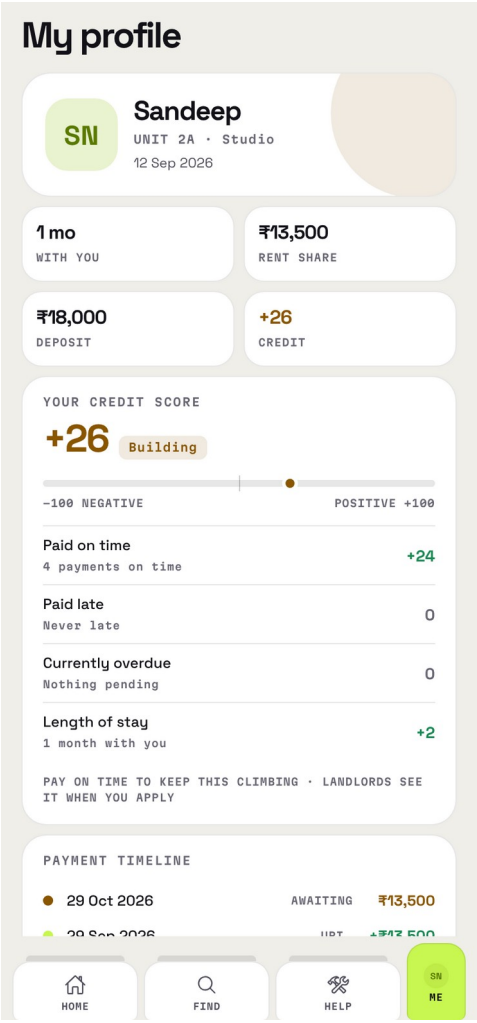


Fig 7.8 — Tenant profile: tenure, rent share, deposit, the behaviour score with its factors, and the payment timeline

7.9 Maintenance

A tenant raises a request with a category, a priority and an optional photograph, and follows it in a thread shared with the landlord, with status changes appearing in sequence alongside the conversation.

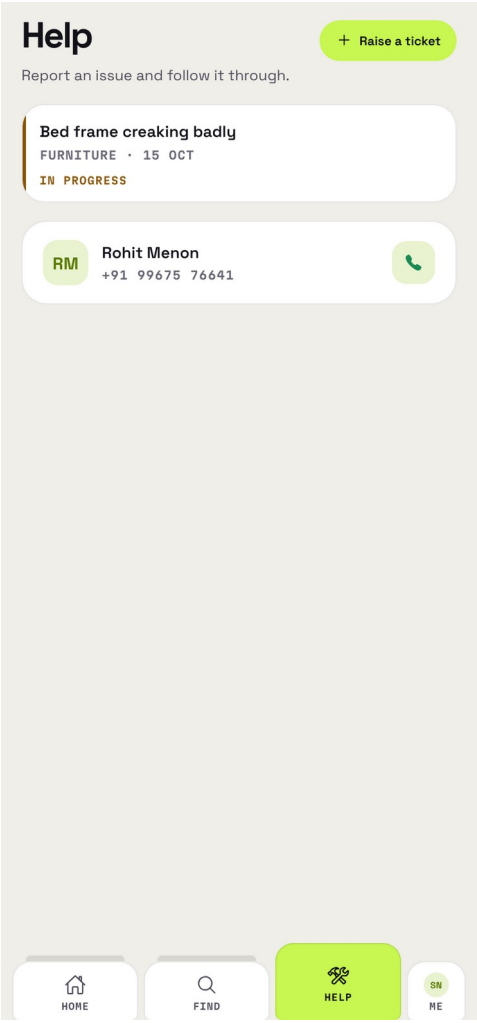


Fig 7.9 — Tenant help screen listing maintenance requests with their status

CHAPTER 8 — CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

This project set out to address a specific and long-standing gap in the independent rental sector: the continued dominance of manual record keeping, and the resulting absence of any shared, authoritative account of what a tenant owes and what they have paid. The system built to address it is a working two-sided application, deployed and in use, consisting of a React Native client and a Node and Express API over MySQL, running to 38,536 lines across 124 source files, 87 endpoints, 22 tables and 28 screens.

The design decision that distinguishes it from the alternatives reviewed in Chapter 2 is that the platform never touches the money. Rent moves directly from the tenant to the landlord, by UPI or through the landlord's own merchant account, and the platform's role is limited to stating the correct amount and destination and to recording what happened. This keeps the operator outside the payment-aggregator perimeter, costs both parties nothing, and leaves settlement where the landlords consulted during this project wanted it. The price of that decision is that the system cannot observe the money it does not handle, and the confirmation model exists precisely to pay that price honestly rather than to pretend it does not apply.

The verification work is the part of the project that produced the most useful findings. Exercising all 87 endpoints against a real database and the compiled application against a real browser engine surfaced five defects that unit tests over mocked dependencies had not, including a cross-account authorisation hole that allowed one landlord's tenant to be written into another landlord's room. Two further defects were found in the test harnesses themselves. The experience supports a conclusion that is well established in the literature but is easy to ignore in a student project: tests that assert on the absence of a string, and systems that are only ever tested against their own mocks, give false confidence.

More broadly, the project demonstrates that an independent landlord does not have to choose between a paper register and surrendering custody of their rent to a platform. A system can give both sides a complete and shared record while leaving the money exactly where it already flows.

8.2 Limitations of the Present Work

The limitations are stated plainly, since each follows from a design decision rather than from an oversight. The application cannot observe money: it never sees a bank balance, so a UPI payment stays a claim until the landlord confirms it, and for landlords who do not connect a gateway the confirmation step is structural rather than incidental. Revocation lags by up to two hours on the access path, because closing that gap completely would require a database lookup on every authenticated request.

Reuse detection can produce a false positive: a client that stores a new access token but fails to store the refresh token issued with it will present a spent token and have every session revoked. The client-side write is transactional, but a device that loses its storage halfway through would still reach this state. SMS remains unconfigured, because phone verification requires a telecom registration that has not been completed; email and push are both live. No iOS build has ever been produced; the bundle identifier is unset and the project has only produced Android artifacts.

Gateway onboarding is heavy. A landlord connecting Razorpay faces that gateway's own verification process, realistically two to five working days, and although the application now states this before they begin, it remains the largest single piece of friction in the product. Finally, there is no automated regression gate: the suites exist and pass, but nothing runs them on push, so verification remains a manual step somebody has to decide to take.

8.3 Future Enhancement

Several directions have been identified, some specified in detail and some larger in scope.

The immediate items are small and well understood. Reply notifications on maintenance threads are incomplete: a tenant replying to a repair request currently notifies nobody, although the landlord-to-tenant direction already works. The landlord's own payment-entry path inserts the payment and updates the due date in two separate statements where the tenant-facing path is already transactional, and that asymmetry should be removed. The test suites are reproducible and fast enough to run on every push, so wiring them into continuous integration is configuration work rather than development work. An iOS build needs a bundle identifier and a first run through the build pipeline. A landlord announcement feature, a one-to-many message to everyone in a property, was specified

during the notification work and never built, although the alert infrastructure that would carry it already exists.

The larger directions require more thought. Split settlement through a gateway's linked-account facility would let landlords onboard from inside the application, with the gateway performing their verification, which would remove the onboarding friction named above. The cost is that funds would then pass through a platform account, which is a change of regulatory category rather than of configuration, and the 2025 directions restrict such arrangements in ways that call for professional advice rather than inference. It is worth revisiting when the user base justifies it, and not before.

Monetisation, if it is ever pursued, should avoid entering the money path. The Namma Yatri precedent applies almost directly: a flat per-landlord subscription keeps the operator outside the payment flow, whereas a percentage of each transaction begins to resemble aggregation structurally. A related legal nuance is that models of this kind rest on selling access to software rather than on setting the price of the underlying service, and in TenantPro the landlord sets the rent, which is a position worth preserving deliberately.

Finally, deliverability and messaging both have identified work. Transactional mail currently sends from a consumer address through a provider's HTTPS interface; a verified sending domain with the usual authentication records would materially improve inbox placement for about a thousand rupees a year. Phone verification needs an Indian telecom registration before any operator will carry template traffic.

REFERENCES

[1] Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[2] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

[3] Elmasri, R., & Navathe, S. B. (2017). Fundamentals of Database Systems (7th ed.). Pearson.

[4] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

[5] Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley Professional.

[6] Nielsen, J. (1993). Usability Engineering. Morgan Kaufmann Publishers.

[7] Reserve Bank of India. Guidelines on Regulation of Payment Aggregators and Payment Gateways, and subsequent amendments. Retrieved from https://www.rbi.org.in/

[8] Commentary on the Regulation of Payment Aggregators Directions, 2025 — industry analysis of the revised framework for payment aggregators.

[9] Razorpay Developer Documentation. Payment Links API. Retrieved from https://razorpay.com/docs/payments/payment-links/

[10] Razorpay Developer Documentation. API Keys, and the website verification required for live mode. Retrieved from https://razorpay.com/docs/

[11] Razorpay. Route: split settlement and linked accounts. Retrieved from https://razorpay.com/docs/payments/route/

[12] Razorpay. KYC documents required by business type. Retrieved from https://razorpay.com/docs/

[13] National Payments Corporation of India. UPI Linking Specification and Deep Linking Guidelines. Retrieved from https://www.npci.org.in/

[14] Namma Yatri. Zero-commission direct-payment model — public documentation of the open mobility initiative. Retrieved from https://nammayatri.in/

[15] Internet Engineering Task Force. RFC 7519: JSON Web Token (JWT). Retrieved from https://www.rfc-editor.org/rfc/rfc7519

[16] Internet Engineering Task Force. OAuth 2.0 Security Best Current Practice, including refresh token rotation and replay detection. Retrieved from https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics

[17] National Institute of Standards and Technology. SP 800-38D: Recommendation for Block Cipher Modes of Operation — Galois/Counter Mode (GCM). Retrieved from https://csrc.nist.gov/

[18] Expo Documentation. Expo Updates and EAS Build. Retrieved from https://docs.expo.dev/

[19] React Native Documentation. Retrieved from https://reactnative.dev/

[20] Node.js Foundation. Node.js Official Documentation. Retrieved from https://nodejs.org/en/docs/

[21] Express.js. Express Web Framework Documentation. Retrieved from https://expressjs.com/

[22] Oracle. MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/

[23] Microsoft. Playwright Documentation — browser automation for end-to-end testing. Retrieved from https://playwright.dev/

Note on secondary sources. Several of the regulatory points in this report rest on consultancy and industry commentary rather than on the Reserve Bank of India's own text. Where the report leans on them, particularly for the net-worth thresholds and the restrictions on split settlement, the claims should be checked against the Directions themselves before anybody makes a structuring decision. The code figures, endpoint counts and assertion totals were measured directly from the repository and from live test runs, and are not estimates.